Housing Prices Time Series Project

by Carter Kulm

Contents

1 Abstract

Housing prices are almost always a concern for residents of the U.S. regardless of differing characteristics like age, gender, or occupation. The median price of a house at any point in time signifies the magnitude of demand for housing, in turn serving as a barometer for the general state of the economy at that time. Thus by attempting to model the median house price over time we can gain insight into the patterns that dominate the market. In this report, I will conduct time series analysis on U.S. housing prices using Box-Jenkins methodology, along the way forecasting more recent data using a chosen model. In order to forecast, however, the process of model selection and fitting must first occur, which requires a detailed examination of the data at hand. First, transformations and differencing will likely be applied to our data in an effort to create a stationary time series. We'll then identify potential models that could represent our housing data, and these models' performance on the data will inform us of the best fits for the data. We will subsequently perform diagnostic tests on the best-performing models to decide which model will best accomplish the task of forecasting. Through this process it became apparent that high volatility seen at certain time periods in the data makes housing prices extremely difficult to model using Box-Jenkins, and non-linear methods may be necessary in order to accurately fit a model to the data. This unpredictable volatility, despite oftentimes contradicting trends that had been developing over long periods, seems to be inevitable given enough time and is what likely prevents analysts from making accurate market forecasts. However, it should be said that the predictions made by our fitted model onto the testing data were relatively accurate given the aforementioned volatility in housing prices.

2 Report

2.1 Introduction

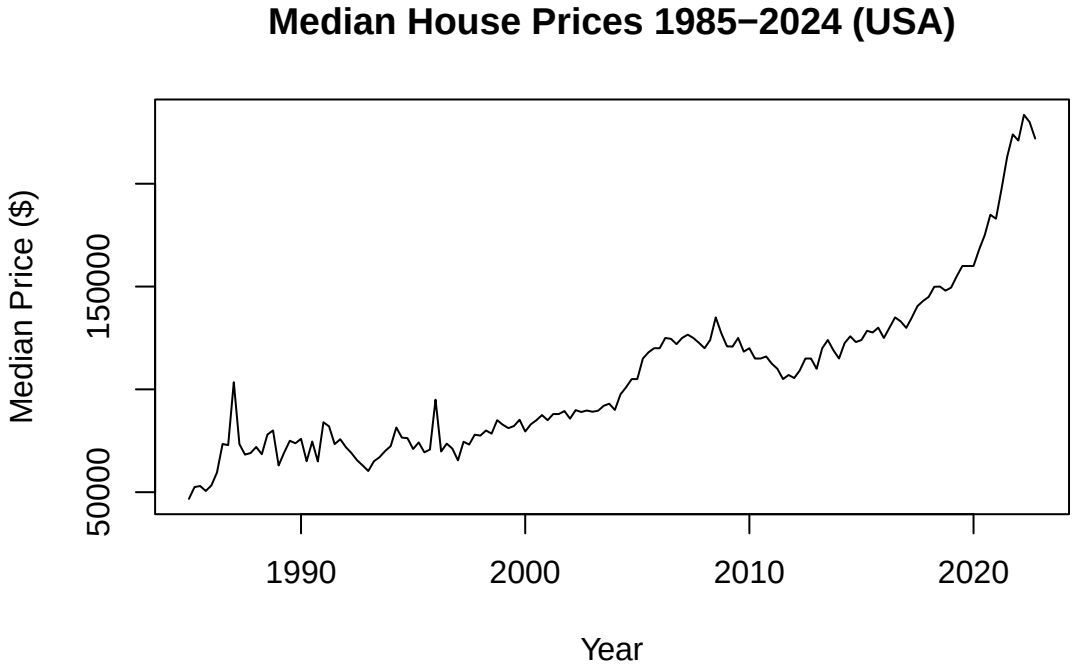
The specific data that will be used in this report's analysis is a dataset of median prices for manufactured homes in the U.S.A. which is updated quarterly by the Federal Housing Finance Agency (FHFA). The data is available for download through this link. The range of the dataset stretches from the first quarter of 1985 to the third quarter of 2024, resulting in 159 observations, a sufficient sample size for our purposes. Housing prices tend to be representative of the financial state of a country, making it interesting to observe the state of the housing market from the recent past. Furthermore, predicting the future of this kind of data may give us insight into what to expect from the economy in the coming years. The general flow of the following report will follow in this order: transforming data, identifying potential models, fitting these models, comparing performances between models, conducting diagnostics on a select number of models, and ultimately forecasting with a single model. Early in the process mentioned above, it became apparent that certain periods of fluctuation in the data represented unstable variance, and would prevent a very accurate model from being fit. While nonlinear time series analysis would likely be required to capture the intricacies of the data, the Box-Jenkins methodology used for analysis in this report produced a relatively robust model and forecast for the given data. All analysis will be conducted using my local RStudio server.

2.2 Splitting Data

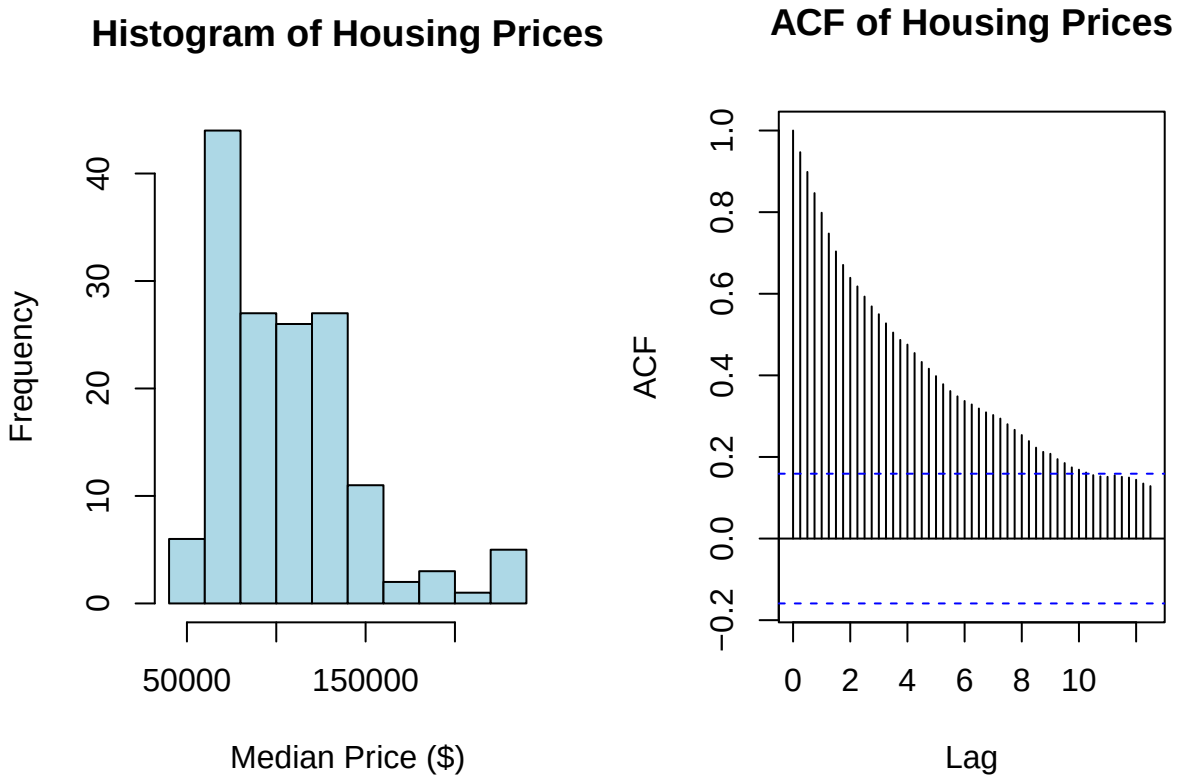
```
# training set
prices_train <- ts(prices[c(1:152)], frequency = 4,
                   start = c(1985, 1), end = c(2022, 4))
# testing set
prices_test <- ts(prices[c(153:159)], frequency = 4,
                  start = c(2023, 1), end = c(2024, 3))
```

We set aside 7 observations at the end of our dataset aside to serve as validation for our time series model which we will train on the first 152 data points ranging from 1985 to 2022.

2.3 Plotting Time Series Data



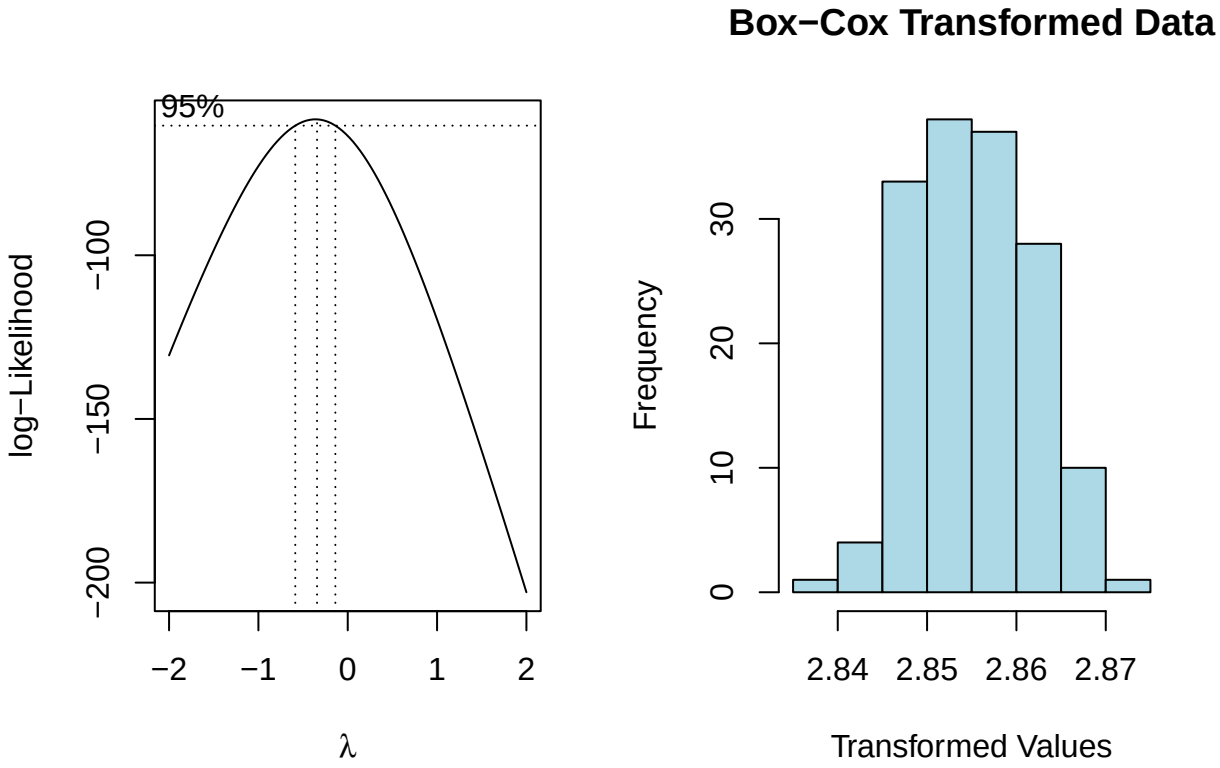
Looking at our time series data, there appears to be a general upwards trend across the years, though there are more volatile periods that violate this trend, as seen around 1990 or the late 2000s. No seasonality can be seen by looking at the data itself, but the ACF and PACF plots can also be helpful to detect a seasonal component. To confirm that our data is non-stationary, we can assess a histogram of the housing price values as well as an ACF plot.



The histogram of median house price shows a right skewness, and the ACF plot displays very large ACF values at earlier lags in the data, both of which telling us that our data is non-stationary. Thus we will proceed to perform transformations.

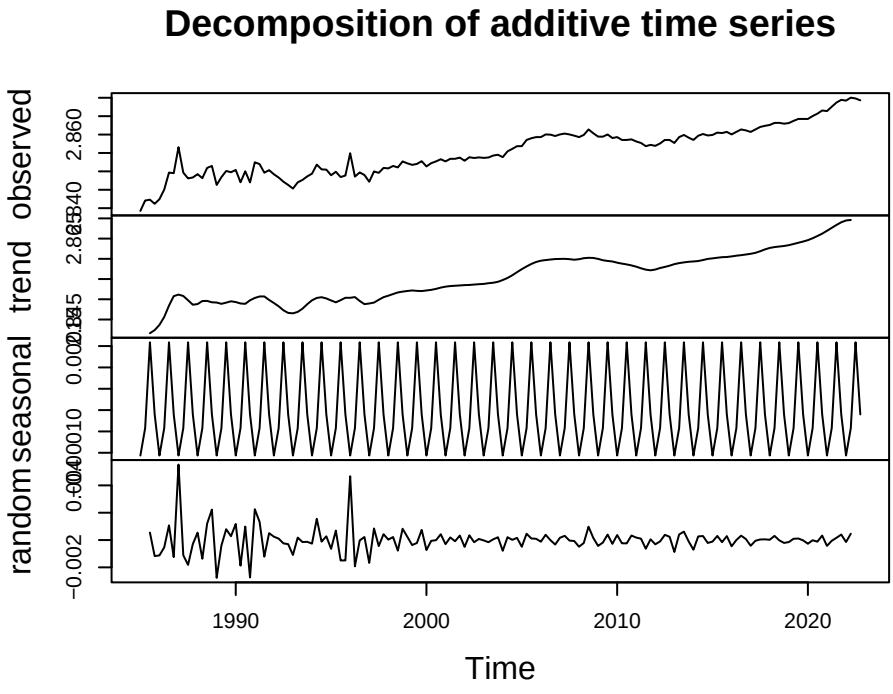
2.4 Transformations

Below we will check to see if a Box-Cox transformation would fit our data well. If the confidence interval for the lambda value to be used includes 0, a different transformation should be opted for.

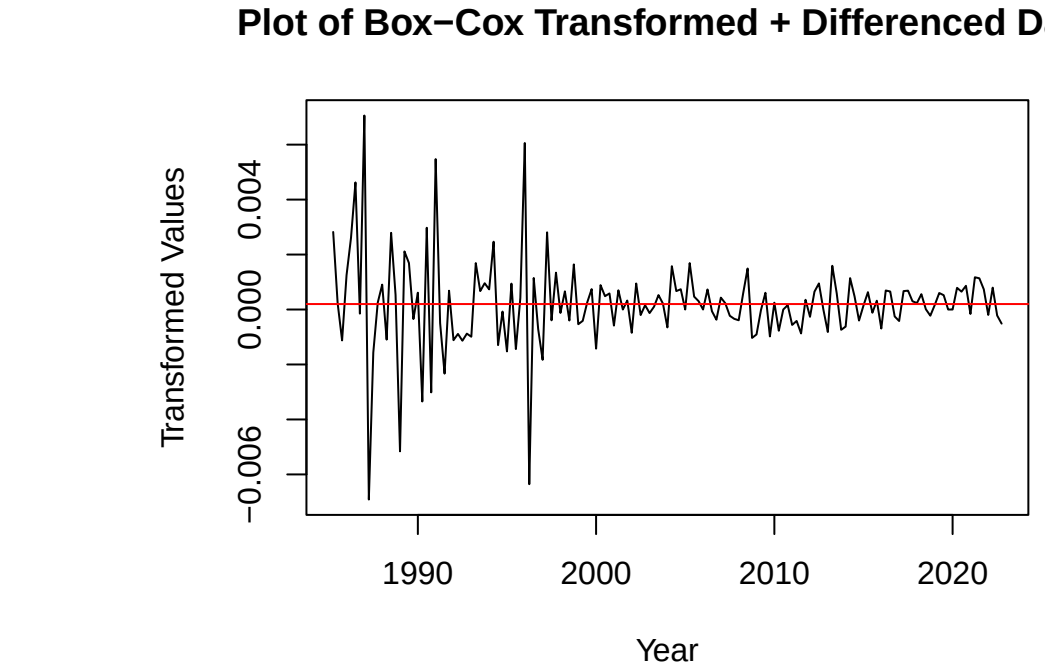


0 narrowly falls outside of the interval of optimal lambda values, so we move forward with the Box-Cox transformation. On the right, the transformed housing price data is displayed in the form of a histogram; the resemblance of the distribution to a Normal distribution indicates that the transformation was successful, especially compared to the original distribution of housing price values.

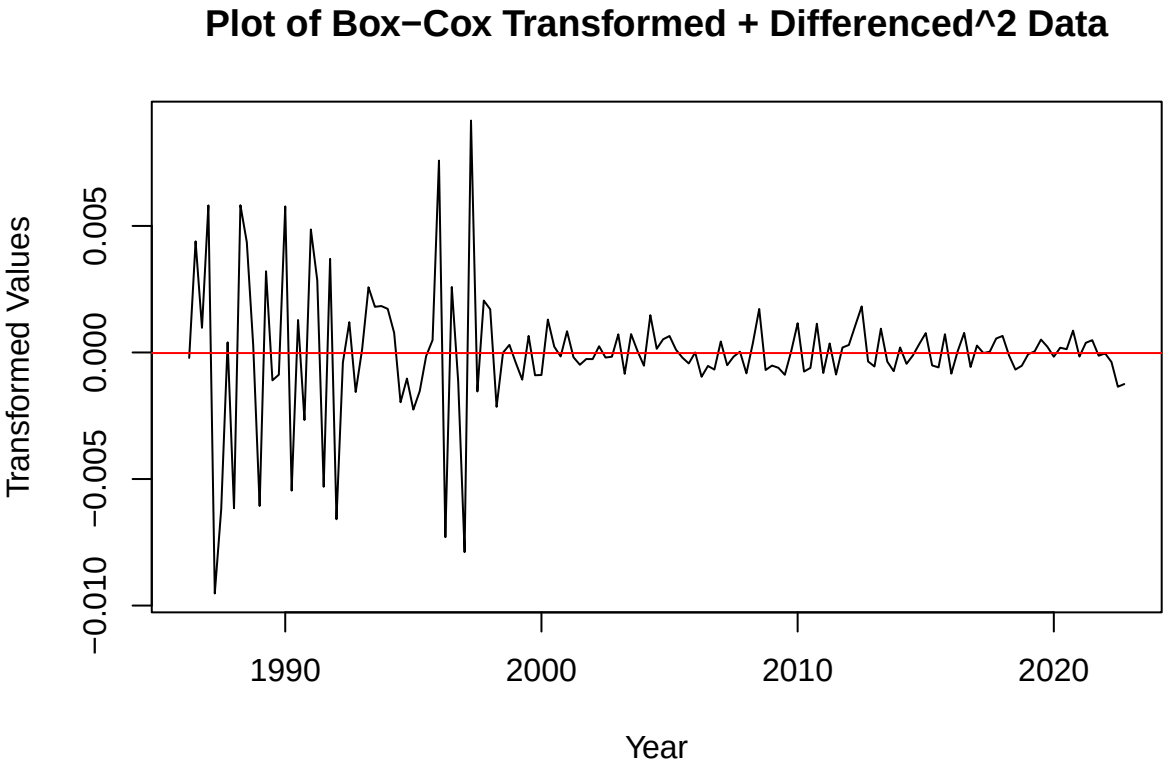
Below we can plot the different components of our now-transformed data to see if any differencing is necessary to remove seasonality and/or trend.



In the "trend" plot, a roughly linear trend is visible, so we will difference at lag 1 to remove this trend, then plot the newly differenced data. The differencing at lag 1 appears to have served its purpose, as our data now looks to have no trend.



The differencing at lag 1 appears to have served its purpose, as our data now looks to have no trend. Now, we can difference the data at lag 4 to see if the seasonality can be removed from the data.



Based on the above plot, it's difficult to determine whether or not the differencing at lag 4 to remove seasonality was successful. Thus we should check below if the variance increased or decreased after the separate differencings.

```
## [1] "Variance of Box-Cox Transformed Data:  4.26093574540937e-05"
```

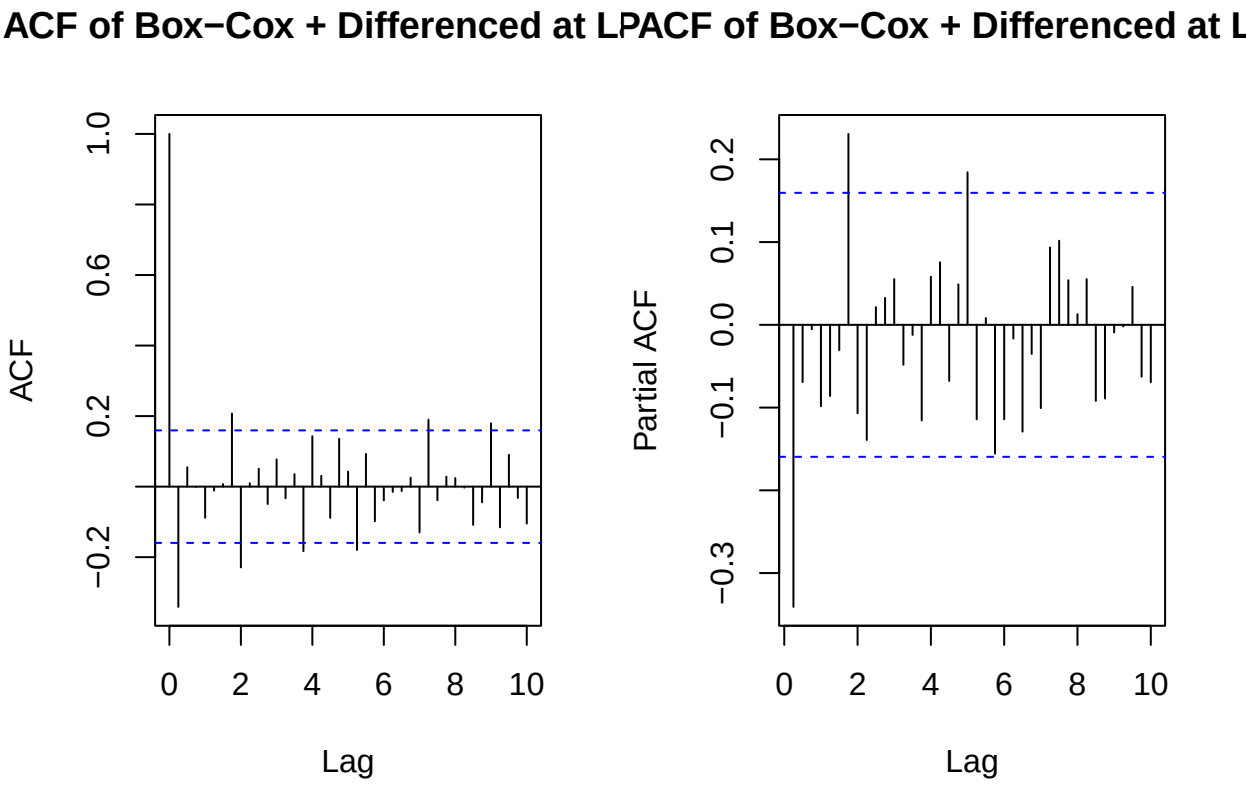
```
## [1] "Variance of Box-Cox + Differenced Data:  2.68580533849777e-06"
```

```
## [1] "Variance of Box-Cox + Twice Differenced Data:  5.93083468657701e-06"
```

We see that the variance of our data decreased after the first differencing to remove trend, but more than doubled after differencing at lag 4 to remove seasonality. Thus it appears that the twice differenced data shows signs of over-differencing, indicating that we should proceed to the model identification stage of the project with the once-differenced data.

2.5 Model Identification

Using the ACF and PACF of our data, we will now identify SARIMA model parameters p, P, q, and Q.



We see potentially significant spikes in ACF at lags of 1, 7, and 8, while the PACF values above show spikes at lag values of 1, 7, and 20. Based on the information that the ACF plot showed spikes at lags 1, 7, and 8, we can now consider p, P, q, and Q values (we already know s = 4, d = 1, D = 0). The seasonal spike seen in the ACF plot at lag 8 indicates that Q = 2 should be tested (in addition to 0 to address the case with no moving average seasonal component), while the spike at lag 7 of the PACF plot tells us that P likely is 0, but it may be worth testing P = 2. Meanwhile, the presence of spikes at lag 1 in both the ACF and PACF plots strongly indicates a q value of 1 and a p value of 0 or 1, respectively. Now we can proceed to the model fitting portion of the project.

2.6 Model Fitting

As established in the last section, we will fit models with combinations of q = 1, Q = 0 or 2, p = 0 or 1, P = 0 or 2, d = 1, D = 0, and s = 4. We will begin with moving average models, testing ones with and without seasonal components.

```
##
## Call:
## arima(x = prices_bc, order = c(0, 1, 1), seasonal = list(order = c(0, 0, 2),
##     period = 4), method = "ML")
##
## Coefficients:
##           ma1     sma1     sma2
##       -0.2966  -0.0694  -0.1706
## s.e.   0.0725   0.0816   0.0799
##
## sigma^2 estimated as 2.331e-06:  log likelihood = 764.74,  aic = -1521.48
```

```
## [1] "AICc of SARIMA(0, 1, 1)(0, 0, 2):  -1521.20474775023"
```

The model with two seasonal moving average components (shown above) produces a slightly lower AICc value than a purely MA(1) model, indicating that it fits the data better. We also see that the first seasonal moving average component of the model has a coefficient whose estimated value lies within 2 standard errors of 0, so we fix its value as 0 and re-check the model.

We see that the modified model with the fixed coefficient set to 0 produces a lower AICc than the original model, so we proceed with the modified model. The next course of action is to test autoregressive models fit to the data, again testing models with and without seasonal components.

```
##
## Call:
## arima(x = prices_bc, order = c(1, 1, 0), seasonal = list(order = c(2, 0, 0),
##     period = 4), method = "ML")
##
## Coefficients:
##           ar1     sar1     sar2
##       -0.3190  -0.1329  -0.2675
## s.e.   0.0774   0.0808   0.0875
##
## sigma^2 estimated as 2.252e-06:  log likelihood = 767.14,  aic = -1526.27
```

```
## [1] "AICc of SARIMA(1, 1, 0)(2, 0, 0):  -1525.99676832318"
```

As was the case with our moving average models, the autoregressive model with the seasonal components slightly outperformed its AR(1) counterpart with an AICc of -1525.997. Additionally, this model produced a lower AICc value than the moving average model seen above. And, we observe that the first seasonal autoregressive component's coefficient estimate lies within 2 standard errors of 0, so we should fix the coefficient as 0.

This time, fixing the coefficient to 0 did not result in a lowered AICc value. Now, we move on to testing mixed models with p = 0 and 1, q = 0 and 1, with P = 2 and Q = 2. Now, we move on to testing models with different combinations of moving average and autoregressive components.

```
##
## Call:
## arima(x = prices_bc, order = c(1, 1, 0), seasonal = list(order = c(2, 0, 2),
##     period = 4), method = "ML")
##
## Coefficients:
```

```
##           ar1    sar1     sar2     sma1    sma2
##       -0.3223  0.3821  -0.8471  -0.6068  0.7883
## s.e.   0.0779  0.1678   0.0792   0.1982  0.1298
##
## sigma^2 estimated as 2.072e-06:  log likelihood = 772.04,  aic = -1532.09

## [1] "AICc of SARIMA(1, 1, 0)(2, 0, 2):  -1531.50458387442"
```

The results of a mixed model is displayed above, and we observe that it produces the lowest AICc value found so far: -1531.505. It's notable that no coefficient included in the model have 0 within 2 standard errors of their estimate, so we proceed to diagnostics with a model that has no fixed coefficients. It should be noted that all models whose AICc values were calculated but not shown in the model fitting section will be included in the code appendix.

2.7 Diagnostics

1) Lowest AICc: $SARIMA(1,1,0) \times (2,0,2)_{12}$: $\nabla_1(1 + 0.3223_{0.0779}B)(1 - 0.3821_{0.1678}B^4 + 0.8471_{0.0792}B^8)BoxCox(X_t) = (1 - 0.6068_{0.1982}B^4 + 0.7883_{0.1298}B^8)Z_t$

$\hat{\sigma}^2 = 2.072\text{e-}06$

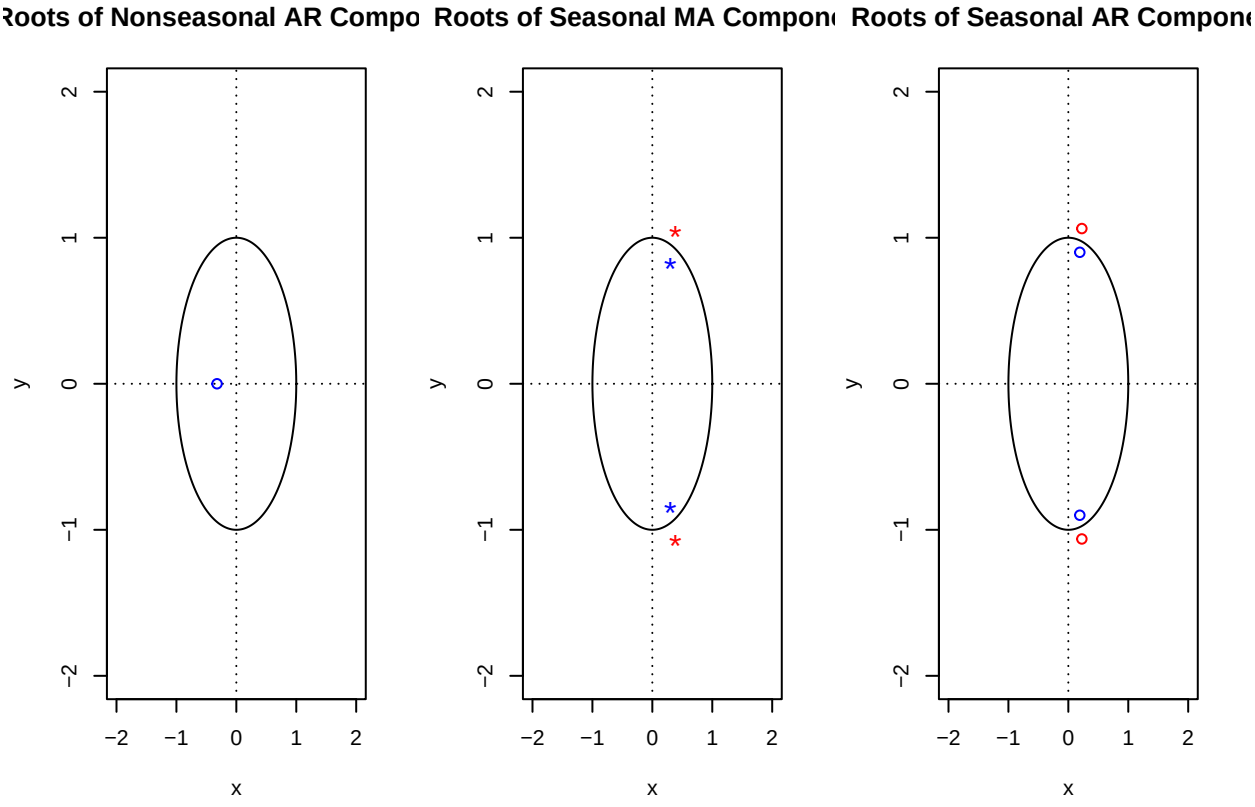
2) Second-Lowest AICc: $SARIMA(1,1,0) \times (2,0,0)_{12}$: $\nabla_1(1 + 0.3190_{0.0774}B)(1 - 0.1329_{0.0808}B^4 + 0.2675_{0.0875}B^8)BoxCox(X_t) = Z_t$

$\hat{\sigma}^2 = 2.252\text{e-}06$

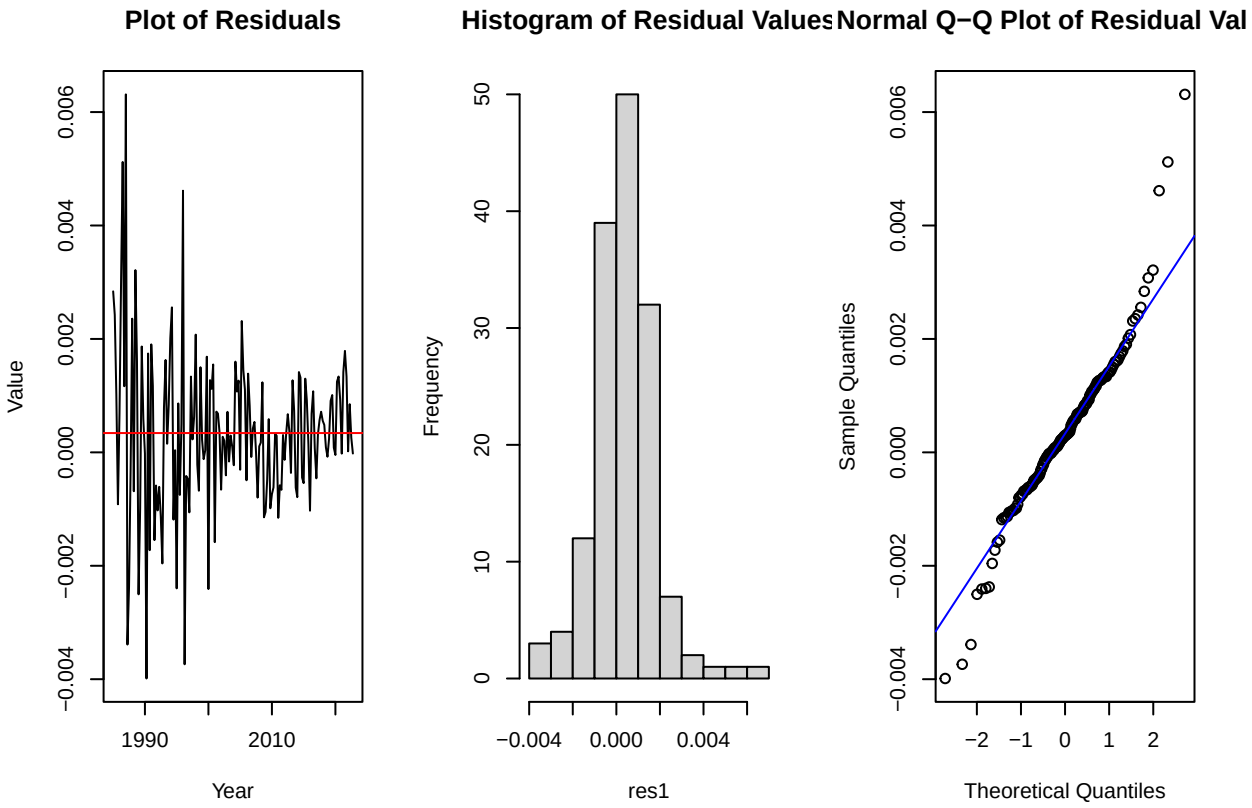
The models that we will perform diagnostic checks on are displayed above, and we will first conduct testing on the model with the lowest AICc value.

2.7.1 Model 1

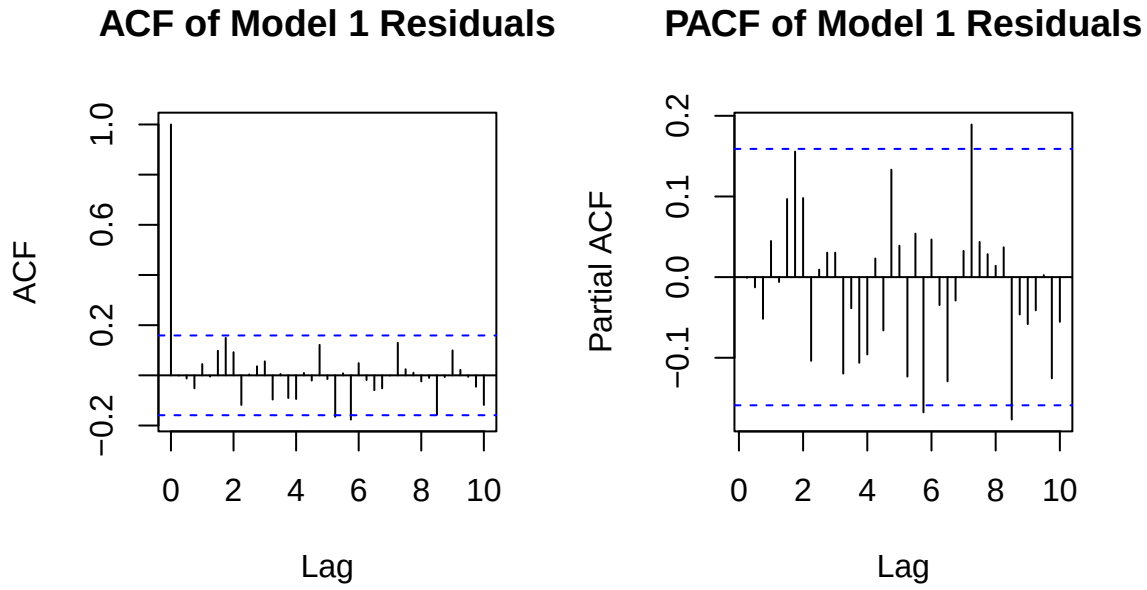
The first diagnostic check will be to ensure that the model is stationary and invertible.



With red points representing unit roots, we observe that all unit roots lie outside of their respective unit circles, making the model both stationary and invertible. Next we must check if the residuals appear to be approximately Normal based off their plotted values, histogram, and QQ-plot.



While the Q-Q plot and histogram of residual values display characteristics similar to Normality, we see that the plot of residual values shows nonconstant variance, and thus does not appear to resemble a White Noise process. While there does not appear to be any visible trend nor seasonality within the values, the spikes in residual values near the start of the time series makes the data look non-Normal. We can now move onto plotting the ACF and PACF of the residual values for our first fit model.



While there are ACF and PACF values which lie outside of the confidence bounds, if we treat the bounds with leniency then none of the larger values should be considered "spikes". Additionally, the large values do not appear to be in any pattern, which indicates independence. Next we will conduct the Shapiro-Wilk test for Normality of residuals.

```
##
##  Shapiro-Wilk normality test
##
## data:  res1
## W = 0.95073, p-value = 3.341e-05
```

As the plotted residual values suggested, our residuals do not pass the Shapiro-Wilk test for Normality. The very small p-value suggests that the residuals are in fact extremely non-Normal. While we have established that the residuals do not resemble a Normal distribution, we can conduct further tests to determine whether the residuals are independent of each other or not. For the Box-Pierce and Ljung-Box tests we will use a lag of $\sqrt{n}$; for our purposes, n = 152, so we use lag = 12. And, we will use fitdf = 5, as we have 5 coefficients included in our first model.

Table 1: Portmanteau Tests for Model 1

	χ^2	df	p-val
Box-Pierce	9.655706	7	0.2089381
Box-Ljung	10.290225	7	0.1727145
McLeod-Li	50.351067	12	0.0000012

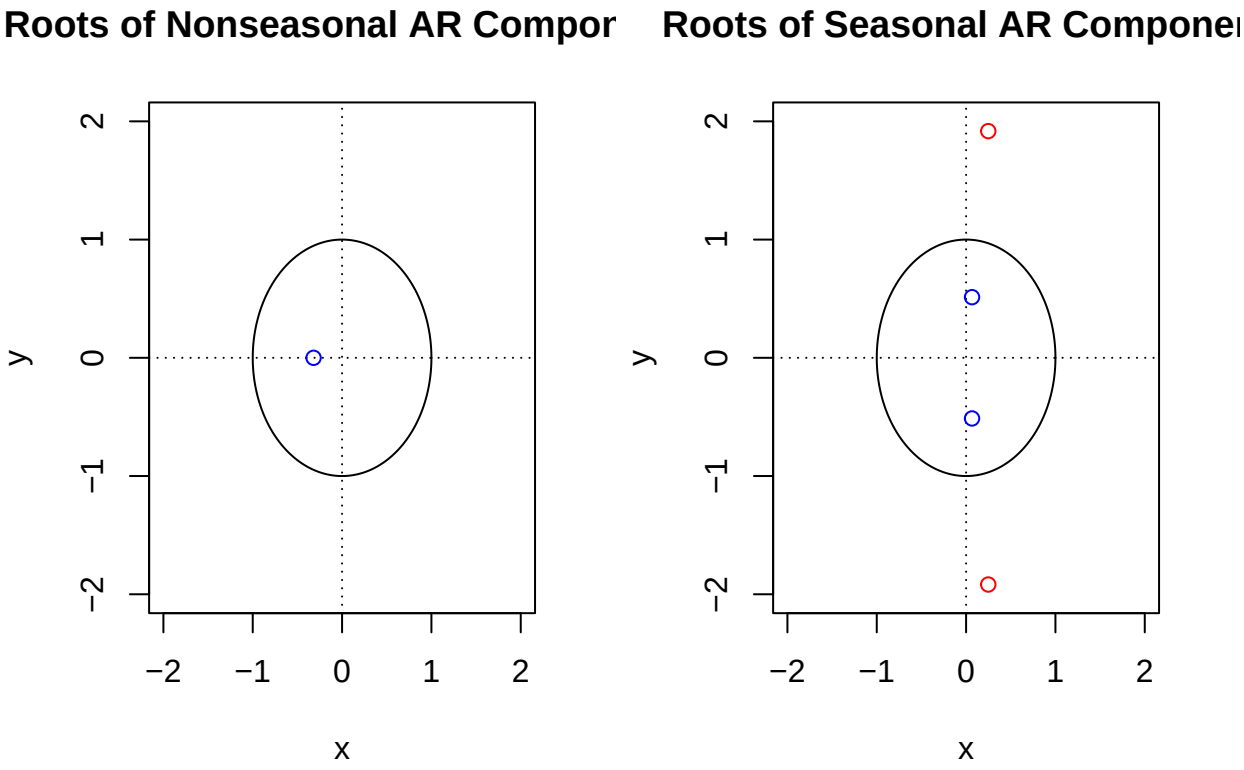
Based on the results, we observe that while our residuals pass the Box-Pierce and Box-Ljung tests, which test for significant autocorrelations in the residuals, it fails the McLeod-Li test with a very small p-value. This also makes sense as we know that the McLeod-Li test can detect non-linear relationships within the residuals, which could be at play with our residuals that exhibit unstable variance. Lastly, we will now fit the residuals to an autoregressive model; if the suggested order of the model is 0, then R is suggesting that the residuals resemble a White Noise process.

```
##
## Call:
## ar(x = res1, aic = T, order.max = NULL, method = c("yule-walker"))
##
##
## Order selected 0  sigma^2 estimated as  2.009e-06
```

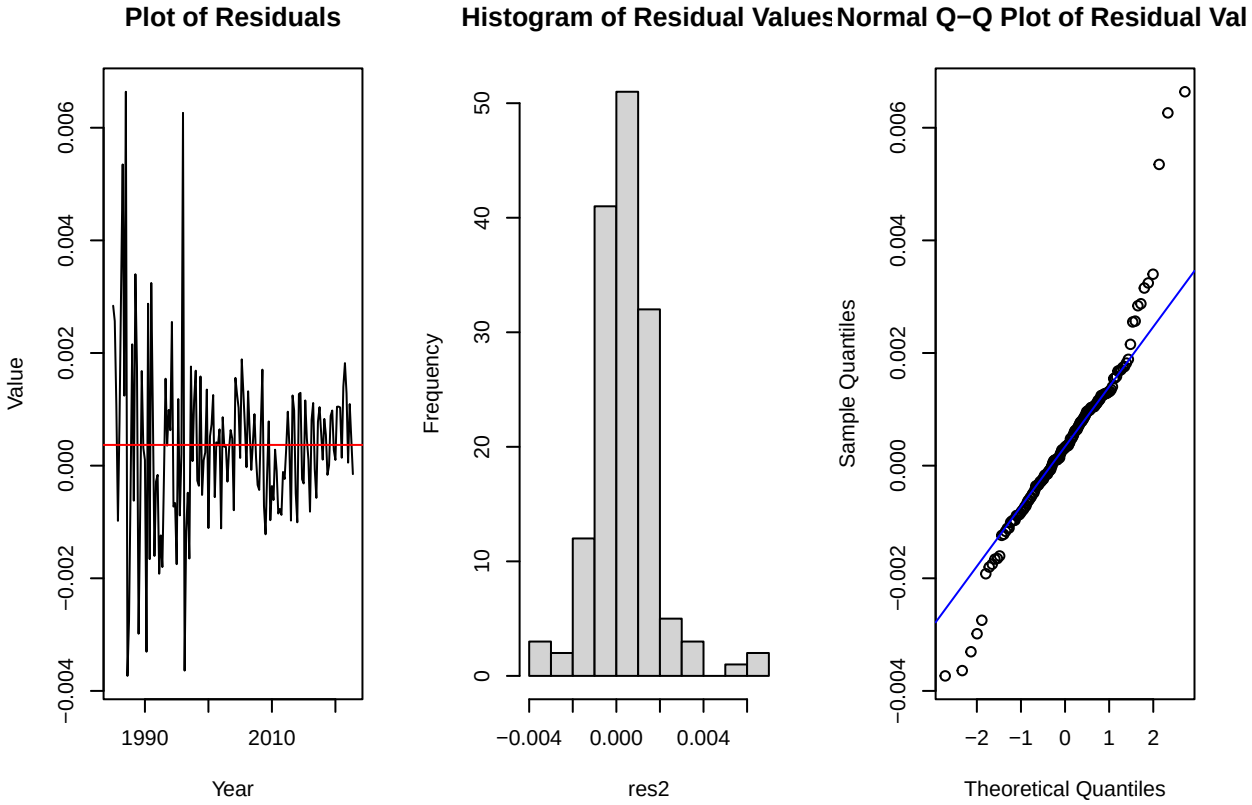
We see that the order selected for the residual autoregressive model is in fact 0, despite the plotted residual values not exactly resembling a White Noise process.

2.7.2 Model 2

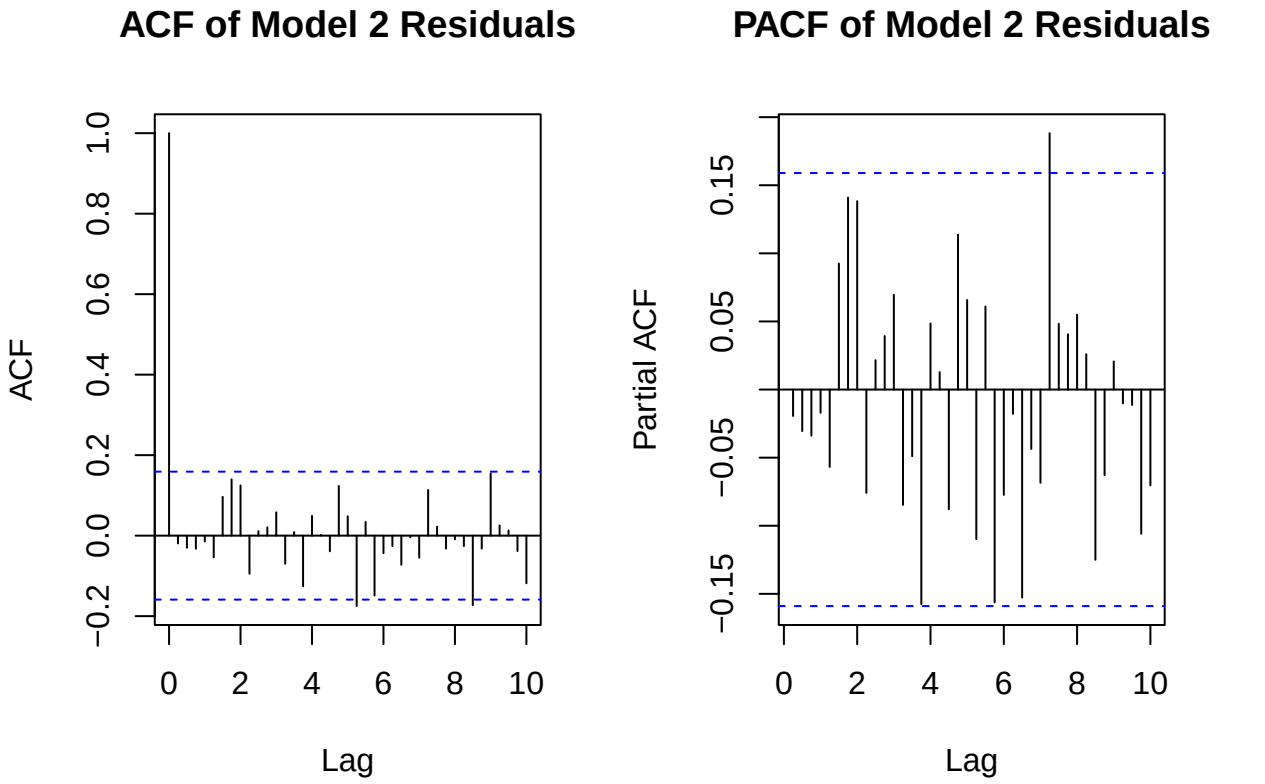
We will repeat the residual diagnostic process for our second fitted model with the second-lowest AICc. We again begin by ensuring that the model is stationary and invertible.



The red points on each plot represent the coordinates of unit roots for each component of the model; luckily, no root looks to lie inside of the unit circle, making the model both stationary and invertible.



The different methods of plotting the residual values for our second fitted model display similar issues to the first model. Likely to the high volatility seen in the early years of our transformed data, the residual values at these years are also very volatile, as seen in the plot of residuals. This is also visible in the histogram of the values, where the spikes in residuals create long tails. We will now display the ACF and PACF plots of the residuals for model 2.



While there are lags at which the ACF or PACF value exceeds the confidence interval's bounds (most notably at lag 29 of the PACF plot), there doesn't look to be any discernable patterns within either plot, nor do there exist any large spikes. The next step in analyzing model 2's residuals will be to undergo statistical tests, beginning with the Shapiro-Wilk test for Normality.

```
##
##  Shapiro-Wilk normality test
##
## data:  res2
## W = 0.92485, p-value = 3.809e-07
```

Predictably, the p-value returned for the Shapiro-Wilk test is below the .05 threshold, indicating a non-Normal set of residual values for model 2. We will now conduct the Box-Pierce, Box-Ljung, and McLeod-Li tests. Due to there being 3 coefficients included in model 2, we will use a fitdf value of 3 for the Box-Piere and Box-Ljung tests.

Table 2: Portmanteau Tests for Model 2

	χ^2	df	p-val
Box-Pierce	9.546586	9	0.3884214
Box-Ljung	10.169304	9	0.3369539
McLeod-Li	43.121662	12	0.0000215

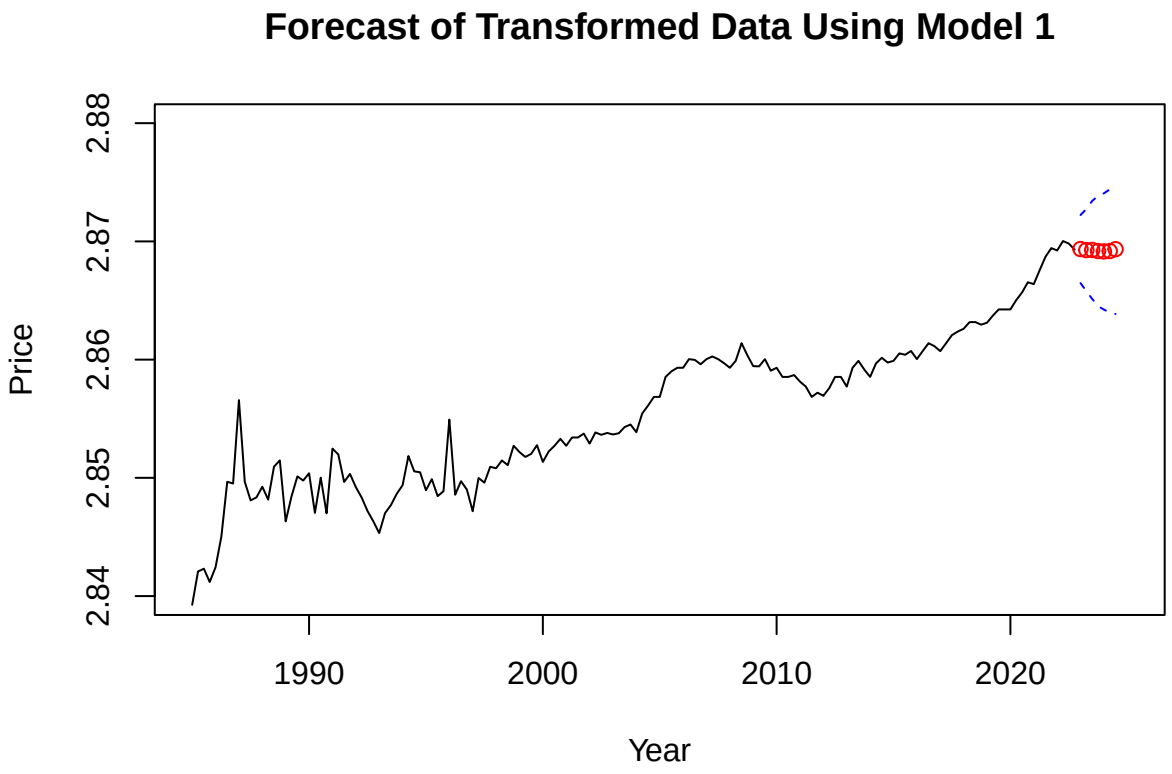
Model 2 behaves similarly to model 1 in regards to these statistical tests, passing the Box-Pierce and Box-Ljung tests but failing the McLeod-Li test, which checks for nonlinear dependencies within the residuals. The last diagnostic for model 2 will be to check the order of autoregressive model that is fit to its residuals.

```
##
## Call:
## ar(x = res2, aic = T, order.max = NULL, method = c("yule-walker"))
##
##
## Order selected 0  sigma^2 estimated as  2.169e-06
```

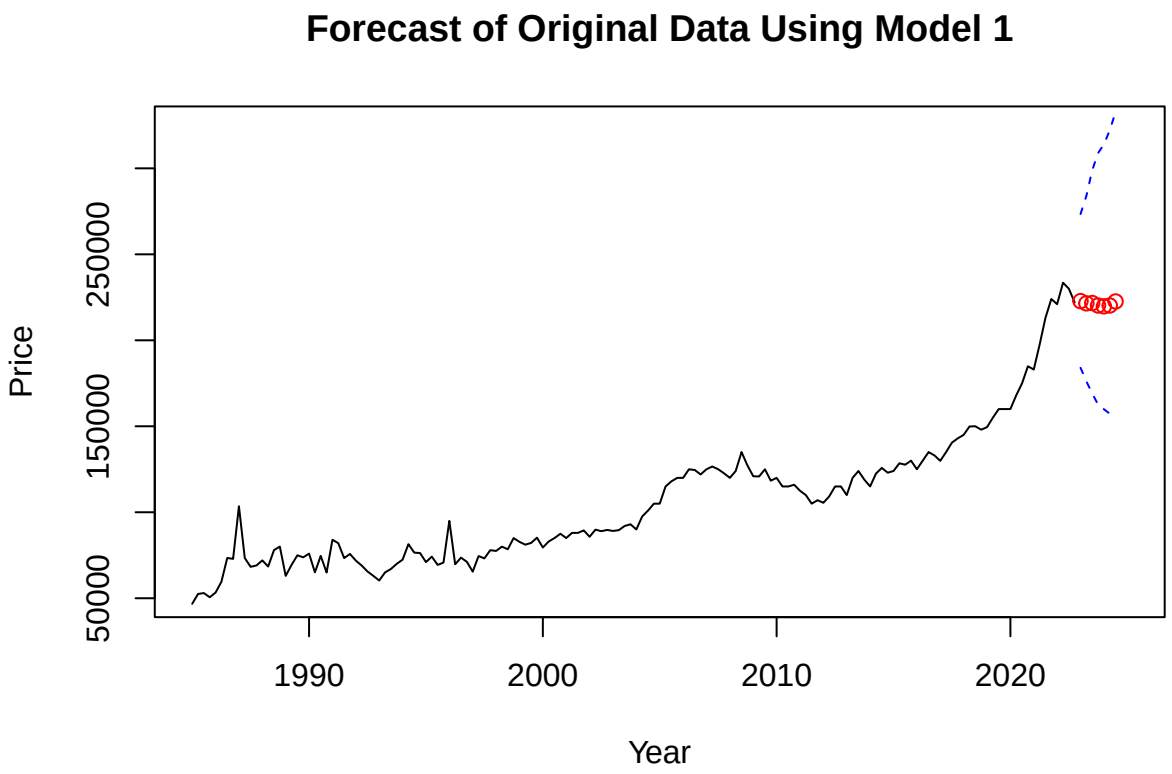
R suggests an autoregressive model of order 0 fit for model 2's residuals, which is equivalent to a White Noise process.

2.8 Forecasting

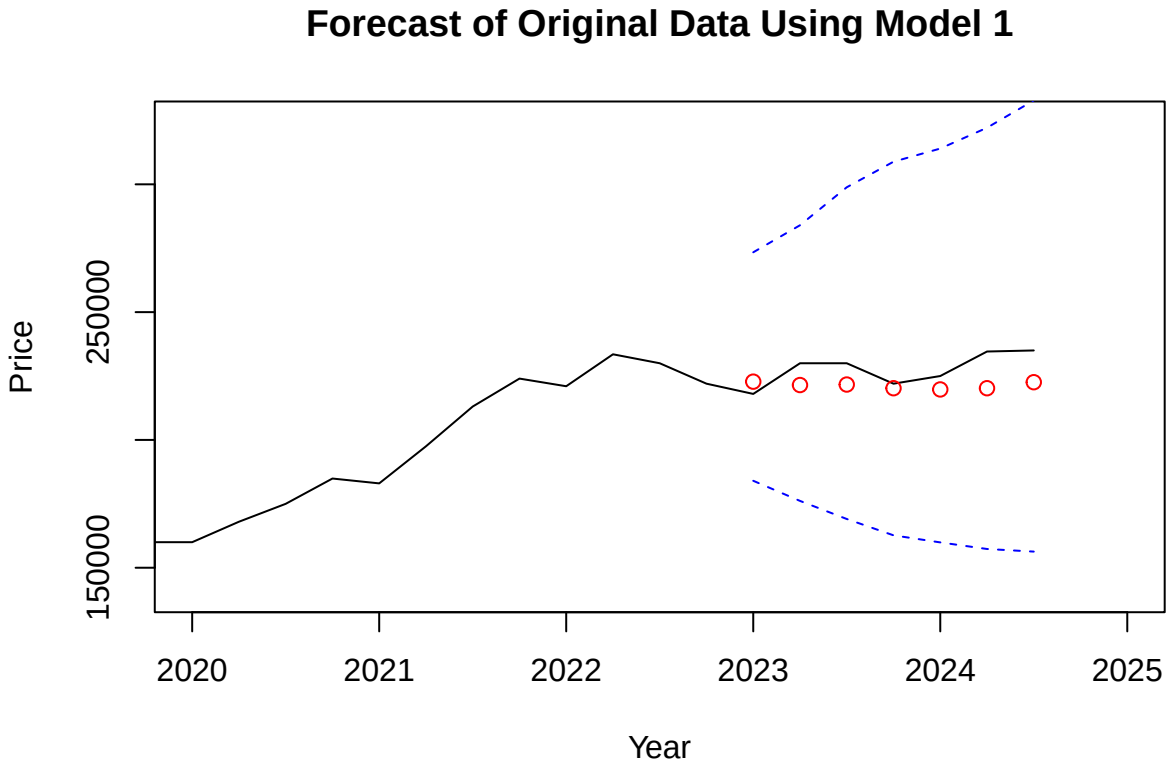
Models 1 and 2 had identical results from the diagnostic stage, but we know from the model fitting step that model 1 possessed the lower AICc value, so we will forecast with it rather than model 2. Thus, the final model that we have chosen to model the Box-Cox transformed data follows a $SARIMA(1,1,0) \times (2,0,2)_{12}$ model, and can be written as such: $\nabla_1(1+0.3223_{0.0779}B)(1-0.3821_{0.1678}B^4+0.8471_{0.0792}B^8)BoxCox(X_t) = (1-0.6068_{0.1982}B^4+0.7883_{0.1298}B^8)Z_t$, with $\hat{\sigma}^2 = 2.072\text{e-}06$. Below we will produce a visual of our Box-Cox transformed data with the last 7 points forecasted using model 1.



Interestingly, the model predicts the current rise in median housing price in the US to slow over the period from quarter 1 of 2023 to quarter 3 of 2024. We observe that each forecasted point lies within the 95% confidence interval of values, thus validating model 1 as a good linear fit to the transformed data. Below, we will produce forecasts for the original, untransformed data to check if the model's predictions hold up.



The forecasted values again lie comfortably within the prediction interval's bounds, indicating a relatively good fit to the original/untransformed data. Thus our model 1 appears to be a good fit for the data. Below we will zoom into the forecasted period to get a better look at the predicted values compared to the true values.



As we see above, the predicted values of the last 7 points which are shown in red do not exemplify the rises and dips in price seen in the original data, shown as the black line. However, the general trend of the data remains relatively stable, which our predictions accurately forecasted.

2.8.1 Conclusion

The final model that was chosen to proceed to forecasting with can be represented as such: $SARIMA(1,1,0) \times (2,0,2)_{12}$, or mathematically: $\nabla_1(1 + 0.3223_{0.0779}B)(1 - 0.3821_{0.1678}B^4 + 0.8471_{0.0792}B^8)BoxCox(X_t) = (1 - 0.6068_{0.1982}B^4 + 0.7883_{0.1298}B^8)Z_t$. Its components represent a model that was differenced once at lag 1 to remove difference, and zero times at lag 4 to remove seasonality, as we saw that differencing to remove seasonality increased variance. Furthermore, we observe that the model contains seasonal autoregressive and moving average components, in addition to a single nonseasonal autoregressive term. In the future, it appears that using nonlinear time series analysis methods would suit the dataset better than our Box-Jenkins methodology, which works best on approximately Gaussian data. Our dataset showed to have both a variance that was dependent upon time (nonstationary) as well as nonconstant variance, making it unfit for such methodology. Despite the non-linear nature of certain behavior included in our data, our model was still able to produce predictions that behaved similarly to the true values of the same time period, which accomplishes our primary goal of forecasting.

I'd like to acknowledge that throughout the process of completing this project, Professor Feldman gave me much guidance along the way, which I very much appreciate.

3 References

House price index datasets: FHFA. FHFA.gov. (2024, May 28). https://www.fhfa.gov/data/hpi/datasets?tab=quarterly-data

4 Appendix

```r
library(tidyverse)
library(readxl)
library(MuMIn)
library(forecast)
library(knitr)
library(kableExtra)
# read in data
housing <- read_xlsx("~/Desktop/PSTAT 174/Project/medians_po_mh.xlsx")
# remove first three rows (headers, column names)
housing <- housing[-c(1:3), ]
# rename variables
colnames(housing) <- c("area", "year", "quarter", "median_price")
# convert price to a numeric variable
housing$median_price <- as.numeric(housing$median_price)
# create time series from "median_price" variable
# frequency = number of observations per cycle (a year is a cycle)
prices <- ts(housing$median_price, frequency = 4,
             start = c(1985, 1), end = c(2024, 3))

# training set
prices_train <- ts(prices[c(1:152)], frequency = 4,
                   start = c(1985, 1), end = c(2022, 4))
# testing set
prices_test <- ts(prices[c(153:159)], frequency = 4,
                  start = c(2023, 1), end = c(2024, 3))

plot(prices_train, main = "Median House Prices 1985-2024 (USA)",
     ylab = "Median Price ($)", xlab = "Year")

par(mfrow = c(1, 2))
hist(prices_train, main = "Histogram of Housing Prices",
     xlab = "Median Price ($)", col = "light blue")
acf(prices_train, lag.max = 50, main = "ACF of Housing Prices")

par(mfrow = c(1, 2))
# calculate the optimal value of lambda for the transformation
t <- 1:length(prices_train)
bc_trans <- MASS::boxcox(prices_train ~ t, plotit = T)
optimal_lambda <- bc_trans$x[which(bc_trans$y == max(bc_trans$y))]
# apply Box-Cox transformation to data
prices_bc <- (1 / optimal_lambda) * (prices_train^optimal_lambda - 1)
# visualize transformed data
hist(prices_bc, main = "Box-Cox Transformed Data", xlab = "Transformed Values",
     col = "light blue")

# retrieve different components of transformed data
components <- decompose(prices_bc)
plot(components)

# difference transformed data once at lag 1
y1 <- diff(prices_bc, lag = 1)
```

```r
plot(y1, main = "Plot of Box-Cox Transformed + Differenced Data",
     xlab = "Year", ylab = "Transformed Values")
abline(h = mean(y1), col = "red")
# difference transformed + differenced data at lag 4
y14 <- diff(y1, lag = 4)
plot(y14, main = "Plot of Box-Cox Transformed + Differenced^2 Data",
     xlab = "Year", ylab = "Transformed Values")
abline(h = mean(y14), col = "red")
paste("Variance of Box-Cox Transformed Data: ", var(prices_bc))
paste("Variance of Box-Cox + Differenced Data: ", var(y1))
paste("Variance of Box-Cox + Twice Differenced Data: ", var(y14))

par(mfrow = c(1, 2))
acf(y1, lag.max = 40, main = "ACF of Box-Cox + Differenced at Lag 1")
pacf(y1, lag.max = 40, main = "PACF of Box-Cox + Differenced at Lag 1")

# q = 1, Q = 2
arima(prices_bc, order = c(0, 1, 1),
      seasonal = list(order = c(0, 0, 2), period = 4), method = "ML")
paste("AICc of SARIMA(0, 1, 1)(0, 0, 2): ",
      AICc(arima(prices_bc, order = c(0, 1, 1),
                 seasonal = list(order = c(0, 0, 2), period = 4), method = "ML"))
      )

# q = 1, Q = 2, Q1 fixed
arima(prices_bc, order = c(0, 1, 1), seasonal = list(order = c(0, 0, 2),
                                                     period = 4),
      fixed = c(NA, 0, NA), method = "ML")
AICc(arima(prices_bc, order = c(0, 1, 1), seasonal = list(order = c(0, 0, 2),
                                                          period = 4),
           fixed = c(NA, 0, NA), method = "ML"))

# p = 1, P = 2
arima(prices_bc, order = c(1, 1, 0), seasonal = list(order = c(2, 0, 0),
                                                     period = 4), method = "ML")
paste("AICc of SARIMA(1, 1, 0)(2, 0, 0): ",
      AICc(arima(prices_bc, order = c(1, 1, 0),
                 seasonal = list(order = c(2, 0, 0), period = 4), method = "ML"))
      )

# p = 1, P = 2, P1 fixed
arima(prices_bc, order = c(1, 1, 0), seasonal = list(order = c(2, 0, 0),
                                                     period = 4),
      fixed = c(NA, 0, NA), method = "ML")
AICc(arima(prices_bc, order = c(1, 1, 0), seasonal = list(order = c(2, 0, 0),
                                                          period = 4),
           fixed = c(NA, 0, NA), method = "ML"))

# p = 1, q = 0, P = 2, Q = 2
arima(prices_bc, order = c(1, 1, 0), seasonal = list(order = c(2, 0, 2),
                                                     period = 4), method = "ML")
paste("AICc of SARIMA(1, 1, 0)(2, 0, 2): ",
      AICc(arima(prices_bc, order = c(1, 1, 0),
```

```
                  seasonal = list(order = c(2, 0, 2), period = 4), method = "ML"))
      )

# models not shown
# q = 1, Q = 0
arima(prices_bc, order = c(0, 1, 1),
      seasonal = list(order = c(0, 0, 0), period = 4), method = "ML")
AICc(arima(prices_bc, order = c(0, 1, 1),
           seasonal = list(order = c(0, 0, 0), period = 4), method = "ML"))
# p = 1, P = 0
arima(prices_bc, order = c(1, 1, 0),
      seasonal = list(order = c(0, 0, 0), period = 4), method = "ML")
AICc(arima(prices_bc, order = c(1, 1, 0),
           seasonal = list(order = c(0, 0, 0), period = 4), method = "ML"))
# p = 0, P = 2
arima(prices_bc, order = c(0, 1, 0),
      seasonal = list(order = c(2, 0, 0), period = 4), method = "ML")
AICc(arima(prices_bc, order = c(0, 1, 0),
           seasonal = list(order = c(2, 0, 0), period = 4), method = "ML"))
# p = 0, q = 1, P = 2, Q = 2
arima(prices_bc, order = c(0, 1, 1),
      seasonal = list(order = c(2, 0, 2), period = 4), method = "ML")
AICc(arima(prices_bc, order = c(0, 1, 1),
           seasonal = list(order = c(2, 0, 2), period = 4), method = "ML"))
# p = 0, q = 0, P = 2, Q = 2
arima(prices_bc, order = c(0, 1, 0),
      seasonal = list(order = c(2, 0, 2), period = 4), method = "ML")
AICc(arima(prices_bc, order = c(0, 1, 0),
           seasonal = list(order = c(2, 0, 2), period = 4), method = "ML"))
# p = 1, q = 1, P = 2, Q = 2
arima(prices_bc, order = c(1, 1, 1),
      seasonal = list(order = c(2, 0, 2), period = 4), method = "ML")
AICc(arima(prices_bc, order = c(1, 1, 1),
           seasonal = list(order = c(2, 0, 2), period = 4), method = "ML"))
# p = 1, q = 1, P = 2, Q = 2, p1 = q1 = 0
arima(prices_bc, order = c(1, 1, 1),
      seasonal = list(order = c(2, 0, 2), period = 4),
      fixed = c(0, 0, NA, NA, NA, NA), method = "ML")
AICc(arima(prices_bc, order = c(1, 1, 1),
           seasonal = list(order = c(2, 0, 2), period = 4),
           fixed = c(0, 0, NA, NA, NA, NA), method = "ML"))
# p = 1, q = 1, P = 2, Q = 2, p1 = 0
arima(prices_bc, order = c(1, 1, 1),
      seasonal = list(order = c(2, 0, 2), period = 4),
      fixed = c(0, NA, NA, NA, NA, NA), method = "ML")
AICc(arima(prices_bc, order = c(1, 1, 1),
           seasonal = list(order = c(2, 0, 2), period = 4),
           fixed = c(0, NA, NA, NA, NA, NA), method = "ML"))

par(mfrow = c(1, 3))
# plot the roots and inverse roots of the model's characteristic equations
source("plot.roots.R")
plot.roots(polyroot(c(1, 0.3223)), NULL,
```

```
           main = "Roots of Nonseasonal AR Component")
plot.roots(NULL, polyroot(c(1, -0.6068, 0.7883)),
           main = "Roots of Seasonal MA Component")
plot.roots(polyroot(c(1, -0.3821, 0.8471)), NULL,
           main = "Roots of Seasonal AR Component")

par(mfrow = c(1, 3))
# fit model 1 to transformed data
fit1 <- arima(prices_bc, order = c(1, 1, 0),
              seasonal = list(order = c(2, 0, 2), period = 4), method = "ML")
# retrieve residuals from model 1's fit
res1 <- residuals(fit1)
# visualize residuals of model 1
plot.ts(res1, main = "Plot of Residuals", xlab = "Year", ylab = "Value")
abline(h = mean(res1), col = "red")
hist(res1, main = "Histogram of Residual Values")
qqnorm(res1, main = "Normal Q-Q Plot of Residual Values")
qqline(res1, col = "blue")

par(mfrow = c(1, 2))
acf(res1, lag.max = 40, main = "ACF of Model 1 Residuals")
pacf(res1, lag.max = 40, main = "PACF of Model 1 Residuals")

# Shapiro-Wilk
shapiro.test(res1)
# Box-Pierce
boxpierce1 <- Box.test(res1, lag = 12, type = "Box-Pierce", fitdf = 5)
# Box-Ljung
boxljung1 <- Box.test(res1, lag = 12, type = "Ljung-Box", fitdf = 5)
# McLeod-Li
mcleodli1 <- Box.test((res1^2), lag = 12, type = "Ljung-Box", fitdf = 0)
# display results of tests in a table
stats1 <- matrix(
  c(boxpierce1$statistic, boxpierce1$parameter, boxpierce1$p.value,
    boxljung1$statistic, boxljung1$parameter, boxljung1$p.value,
    mcleodli1$statistic, mcleodli1$parameter, mcleodli1$p.value
    ),
  nrow = 3,
  ncol = 3,
  byrow = TRUE
)
colnames(stats1) = c("$\\chi^2$", "df", "p-val")
rownames(stats1) <- c("Box-Pierce", "Box-Ljung", "McLeod-Li")
kable(stats1, caption = "Portmanteau Tests for Model 1", escape = FALSE)

# fit residuals to ar model
ar(res1, aic = T, order.max = NULL, method = c("yule-walker"))

par(mfrow = c(1, 2))
# roots and inverse roots of model 2's characteristic equations
plot.roots(polyroot(c(1, 0.3190)), NULL,
           main = "Roots of Nonseasonal AR Component")
plot.roots(polyroot(c(1, -0.1329, 0.2675)), NULL,
```

```
           main = "Roots of Seasonal AR Component")

par(mfrow = c(1, 3))
# fit model 2 to transformed data
fit2 <- arima(prices_bc, order = c(1, 1, 0), seasonal = list(order = c(2, 0, 0),
                                                             period = 4),
              method = "ML")
# retrieve model 2's residuals
res2 <- residuals(fit2)
# visualize model 2 residuals
plot.ts(res2, main = "Plot of Residuals", xlab = "Year", ylab = "Value")
abline(h = mean(res2), col = "red")
hist(res2, main = "Histogram of Residual Values")
qqnorm(res2, main = "Normal Q-Q Plot of Residual Values")
qqline(res2, col = "blue")

par(mfrow = c(1, 2))
acf(res2, lag.max = 40, main = "ACF of Model 2 Residuals")
pacf(res2, lag.max = 40, main = "PACF of Model 2 Residuals")

# Shapiro-Wilk
shapiro.test(res2)
# Box-Pierce
boxpierce2 <- Box.test(res2, lag = 12, type = "Box-Pierce", fitdf = 3)
# Box-Ljung
boxljung2 <- Box.test(res2, lag = 12, type = "Ljung-Box", fitdf = 3)
# McLeod-Li
mcleodli2 <- Box.test((res2^2), lag = 12, type = "Ljung-Box", fitdf = 0)
# display results of tests in a table
stats2 <- matrix(
  c(boxpierce2$statistic, boxpierce2$parameter, boxpierce2$p.value,
    boxljung2$statistic, boxljung2$parameter, boxljung2$p.value,
    mcleodli2$statistic, mcleodli2$parameter, mcleodli2$p.value
    ),
  nrow = 3,
  ncol = 3,
  byrow = TRUE
)
colnames(stats2) = c("$\\chi^2$", "df", "p-val")
rownames(stats2) <- c("Box-Pierce", "Box-Ljung", "McLeod-Li")
kable(stats2, caption = "Portmanteau Tests for Model 2", escape = FALSE)

# fit model 2's residuals to ar model
ar(res2, aic = T, order.max = NULL, method = c("yule-walker"))

# predict 7 points ahead of fit1's data (Box-Cox transformed data)
bc_pred <- predict(fit1, n.ahead = 7)
# calculate upper and lower bounds of 95% confidence interval
upper_pred <- bc_pred$pred + (2 * bc_pred$se)
lower_pred <- bc_pred$pred - (2 * bc_pred$se)
# create time series data from predictions
pred_ts <- ts(data = bc_pred$pred,
              start = c(2023, 1), end = c(2024, 3), frequency = 4)
```

```r
ts.plot(prices_bc, xlim = c(1985, 2025), ylim = c(2.84, 2.88),
        main = "Forecast of Transformed Data Using Model 1",
        xlab = "Year", ylab = "Price")
lines(upper_pred, col = "blue", lty = "dashed")
lines(lower_pred, col = "blue", lty = "dashed")
points(pred_ts, col = "red")

# retrieve the inverse Box-Cox transformed forecasted values
og_pred_ts <- InvBoxCox(bc_pred$pred, lambda = optimal_lambda)
# calculate upper and lower bounds of 95% C.I. for original data forecasts
U = InvBoxCox(upper_pred, optimal_lambda)
L = InvBoxCox(lower_pred, optimal_lambda)
ts.plot(prices_train, xlim = c(1985, 2025), ylim = c(50000, 325000),
        main = "Forecast of Original Data Using Model 1",
        xlab = "Year", ylab = "Price")
lines(U, col = "blue", lty = "dashed")
lines(L, col = "blue", lty = "dashed")
points(og_pred_ts, col = "red")

# visualize predicted values on original data compared to true values
ts.plot(prices, xlim = c(2020, 2025), ylim = c(140000, 325000),
        main = "Forecast of Original Data Using Model 1",
        xlab = "Year", ylab = "Price")
lines(U, col = "blue", lty = "dashed")
lines(L, col = "blue", lty = "dashed")
points(og_pred_ts, col = "red")
```